DevOps Engineer

Home Assignment - Kubernetes GitOps Platform

1. Overview

A company is migrating a business-critical application to Kubernetes and wants to establish a standard cloud-native delivery model.

Your task is to build a small reference implementation that demonstrates how you would deploy and operate an application using Kubernetes, GitOps, CI/CD, security best practices, and production-aware design.

The goal is not to build a complete production platform. The goal is to evaluate your engineering judgment, Kubernetes implementation quality, GitOps understanding, security awareness, and ability to explain trade-offs.

---

2. Application Requirements

Create a simple backend service in any language.

The service must expose:

`GET /health`

Returns basic health status.

`GET /nodes`

Returns Kubernetes node information and marks the node where the current pod is running.

The application must use a dedicated Kubernetes `ServiceAccount` with least-privilege RBAC permissions.

3. Kubernetes Requirements

Provide Kubernetes manifests, Helm chart, Kustomize structure, or a combination.

Your solution should include:

- Namespace structure
- Deployment
- Service
- Ingress or documented ingress approach
- ConfigMap usage
- Secret integration approach
- ServiceAccount
- Role or ClusterRole
- RoleBinding or ClusterRoleBinding
- Resource requests and limits
- Liveness and readiness probes
- SecurityContext
- HPA
- PDB
- NetworkPolicy
- Environment-specific configuration

The repository must support:

- `dev`
- `staging`
- `production`

---

4. GitOps and CI/CD Requirements

Flux must be used as the GitOps model.

Direct deployment from CI/CD to the cluster is not acceptable, except for bootstrap or clearly documented exceptional cases.

Include:

- Flux repository structure
- Environment promotion strategy
- CI/CD pipeline definition or example
- Build, test, scan, and push flow
- Image tagging strategy
- Deployment update strategy
- Rollback approach
- Explanation of Flux reconciliation and drift handling

A working CI pipeline is preferred, but a well-documented pipeline design is acceptable if something is not runnable locally.

---

5. Security and Secrets - Extra credit question

Please note: This question is optional and will grant you extra points

Implement or document a secure secrets-management approach.

You may use one of the following:

- External Secrets Operator with Vault, AWS Secrets Manager, GCP Secret Manager, Azure Key Vault, or equivalent
- SOPS with age/GPG
- Another justified approach

You should explain:

- Where secrets are stored
- How workloads consume secrets
- How access is controlled
- How secrets are separated between environments
- How rotation would work in production

Include policy-as-code using a tool of your choice, such as:

- Kyverno
- OPA Gatekeeper
- Conftest
- Checkov
- Datree
- kube-score
- kube-linter

Provide at least a few meaningful policies or checks. Do not include policy-as-code only as a checkbox.

---

6. Monitoring and Logging

You do not need to deploy a full monitoring stack.

Document a production-suitable approach for:

- Application metrics
- Kubernetes workload monitoring
- Cluster-level monitoring
- Centralized logging
- Alerting

- Incident investigation

Include at least three example alerts, such as:

- High error rate
- Pod crash looping
- Deployment unavailable
- HPA unable to scale
- High CPU or memory usage
- Node pressure

---

7. Cloud Production Design

Provide a short production design section explaining how this would run in a real cloud environment.

Choose one major cloud provider or keep the design cloud-agnostic.

Address:

- Cluster architecture
- Public/private networking model
- Ingress, DNS, and TLS
- IAM and workload identity
- Container registry
- Secrets-management integration
- Environment separation
- Audit logging
- Encryption
- Backup and restore
- Scaling and node provisioning
- Cost considerations
- Disaster recovery considerations

8. Deliverables

Submit a Git repository containing:

- Application source code
- Dockerfile
- Kubernetes manifests, Helm chart, or Kustomize structure
- Flux configuration
- `dev`, `staging`, and `production` environment structure
- CI/CD pipeline definition or documented design
- Infrastructure-as-code configuration
- Policy-as-code configuration
- README with deployment instructions
- Architecture diagram
- Assumptions
- Design decisions
- Trade-offs
- Known limitations
- Production recommendations

---

9. Evaluation Criteria

Area	Weight
Kubernetes implementation quality	25%
GitOps and CI/CD design	20%
Security, secrets, and RBAC	20%
Production architecture judgment	15%
Monitoring and logging approach	10%
Documentation and communication	5%
Trade-offs and engineering judgment	5%

---

10. Technical Review

During the interview, you will present your solution, architecture diagram, implementation decisions, trade-offs, and demonstrate your understanding of all technologies used, including any AI-assisted work.